

Architecting an Autonomous Financial Multi-Agent System: Technical Proposal, Risk Mitigation, and Implementation Blueprint

Executive Summary

The paradigm of quantitative finance and retail portfolio management is undergoing a fundamental restructuring, driven by the transition from reactive large language models to autonomous multi-agent systems. The objective of managing a high-conviction satellite portfolio—representing a strictly capped allocation of total wealth alongside a passive core—requires an architecture that transcends the capabilities of conventional conversational chatbots. To consistently outmaneuver market noise, discover alpha-generating equities, analyze non-yielding commodities, and execute rigorous portfolio monitoring, an automated investment engine must be deployed.

This report provides an exhaustive technical blueprint for constructing a read-only, autonomous financial analysis pipeline. Utilizing Google Antigravity—a state-of-the-art agent-first development platform released in late 2025 and significantly updated by March 2026—this architecture leverages specialized artificial intelligence agents acting in concert to execute data ingestion, fundamental valuation, and macroeconomic risk assessment. By integrating deterministic Python logic for mathematical execution with the inferential capabilities of advanced models like Gemini 3 Pro for parameter estimation, the system mathematically isolates the intrinsic value of diverse asset classes.

Crucially, this document responds to the requirement for a critical evaluation of early-stage architectural proposals. It exhaustively examines the systemic vulnerabilities of deploying artificial intelligence in financial workflows, addressing the documented optimistic bias in macroeconomic reasoning, the fragility of automated data ingestion, and the absolute necessity of the Generator-Critic coordination pattern. Furthermore, the report establishes a comprehensive compliance framework aligned with the European Union's Artificial Intelligence Act and the strict 2026 guidelines issued by the Spanish Data Protection Authority (AEPD) regarding agentic autonomy. Finally, a phased implementation plan is proposed, adhering to the principle of starting with simple, single-agent verification before scaling to a complex, multi-agent swarm.

Critical Evaluation of the Proposed Architecture

The initial architectural proposal centered on utilizing a Python-based multi-agent system, orchestrated via frameworks like LangGraph, interfacing with the Interactive Brokers Flex Web

Service (Option B), and utilizing a large language model to estimate parameters for fundamental valuation. While structurally sound as a high-level concept, a rigorous technical critique reveals several severe operational vulnerabilities, architectural weaknesses, and compliance gaps that must be mitigated before deployment in a production environment.

Weaknesses in the Ingestion and Data Bridging Layer

The proposal heavily endorsed the Interactive Brokers (IBKR) Flex Web Service for automated portfolio ingestion due to its avoidance of the daily restart requirements associated with the live IB Gateway. However, relying entirely on automated static reports introduces distinct failure modes. The IBKR Flex Web Service relies on XML and CSV generation which is notoriously susceptible to formatting anomalies. For example, international equities often report values using varying decimal and comma separators depending on the exchange of origin. A naive Python CSV parser that does not explicitly force locale-aware formatting will inevitably ingest malformed floating-point numbers, leading to catastrophic downstream errors in valuation calculations.¹ Furthermore, the Flex Web Service endpoints are subject to rate limits and occasional generation failures (such as the documented Error 1001: Statement could not be generated).¹ An architecture that simply requests the file and immediately attempts parsing without implementing exponential backoff protocols and retry logic will result in brittle pipelines that crash during routine weekend updates.

To mitigate this, the data bridging layer must be decoupled from the analysis layer through a formalized semantic abstraction. Best practices in 2026 mandate the use of a federated data architecture.² Rather than having the AI agent read raw CSV files directly, the Python backend must ingest the IBKR Flex Query, validate the data types using strict schemas (such as those provided by Pydantic), and expose this governed data to the agents via a Model Context Protocol (MCP) server. This ensures the agents only interact with standardized, error-free JSON payloads, entirely eliminating formatting-induced hallucinations.

Cognitive Vulnerabilities and Optimistic Bias in Valuation

The initial proposal suggested utilizing models like Gemini 3.1 Pro to extract forward-looking parameters (such as capital expenditure guidance and revenue growth) from earnings transcripts to feed into a Discounted Cash Flow (DCF) algorithm. The critical weakness in this approach is the well-documented cognitive biases inherent in large language models.

An extensive 2026 empirical study by the International Monetary Fund (IMF) assessing the capabilities of advanced models in macroeconomic analysis revealed a systemic "optimistic bias".³ The models consistently assigned higher, less-dispersed ratings to economic outlooks than human experts and demonstrated severe overconfidence, self-assessing their certainty at 81% to 88% even when their analysis of complex, open-ended macrofinancial issues was fundamentally flawed.³ Furthermore, research into LLM decision-making confirms that models are highly susceptible to the Framing Effect and Status Quo Bias, especially under increased cognitive load when processing highly complex dialogues or massive context windows.⁴

Relying on a single Analyst agent to extract growth parameters will inevitably result in inflated intrinsic value calculations, completely neutralizing the margin of safety required for value investing. To mitigate this, the architecture must implement a rigorous adversarial framework, specifically the Generator-Critic pattern, which will be detailed in the multi-agent topology section.

Cost Control and Token Exhaustion Risks

The proposal failed to account for the exponential token consumption associated with multi-agent systems. Deploying a network of agents that continuously pass the full context of ten years of 10-K filings and global supply chain reports between one another leads to context saturation. Research from 2026 indicates that multi-agent systems can consume up to fifteen times more tokens than single agents, rapidly draining API budgets while simultaneously causing "context rot," where the model becomes confused by the massive volume of irrelevant data retained in its working memory.⁵

The mitigation strategy requires strict token budgeting and progressive disclosure architectures. Data pinning must be implemented during the development and testing phases, allowing the orchestrator to reuse previous API outputs rather than continuously polling live, token-heavy endpoints while iterating on the downstream valuation logic.⁷ Furthermore, the system must match the model size to the specific task complexity; simple portfolio state validation should be routed to smaller, highly efficient models, reserving the massive parameter models exclusively for complex reasoning and parameter extraction.⁶

Antigravity IDE: The Foundational Agent-First Platform

Constructing a robust, self-correcting financial orchestrator requires an environment designed explicitly for asynchronous agent coordination rather than sequential text generation. Google Antigravity, introduced in late 2025 and refined to version 1.20.5 by March 2026, serves as the optimal foundational platform for this endeavor. Antigravity operates as an agentic Integrated Development Environment (IDE) that fundamentally shifts the software development and workflow orchestration paradigm.⁸

The Dual-Interface Architecture and Mission Control

Traditional approaches to building financial automation rely on external script execution and conversational sidebars, which are highly susceptible to linear failure modes. Antigravity mitigates this by bifurcating the workspace into two distinct, native interfaces. The Editor View functions as a traditional, AI-enhanced environment for synchronous programming tasks, providing the human architect with deterministic control over the underlying Python algorithms.⁸

Conversely, the Manager Surface acts as a dedicated "Mission Control" dashboard. This interface is engineered for high-level orchestration, enabling the developer to spawn, monitor, and interact with multiple specialized agents operating asynchronously across different

workspaces.⁸ In the context of a weekly portfolio workflow, the Manager Surface allows the user to deploy a discovery-focused agent to execute automated web searches for emerging sector catalysts, while a concurrent analytical agent interfaces with a local database of fundamental market data, and a third agent executes the Interactive Brokers API script to reconcile current portfolio holdings.¹³ The integration of a built-in headless browser and terminal execution allows these agents to autonomously run scripts, verify API payloads, and test logic without synchronous human intervention.⁹

Artifact-Based Verification Systems

A critical weakness in early autonomous systems was the opacity of their execution chains. Human operators were forced to audit thousands of lines of raw tool-call logs to verify the logic behind a financial recommendation. Antigravity resolves this through an Artifact-based verification system. Instead of generating transient chat text, the agents produce tangible, structured deliverables.⁸

These Artifacts manifest as markdown implementation plans, JSON schema representations of valuation targets, and visual layout charts. This architectural shift ensures that the output of the weekly financial workflow is a highly structured, easily auditable HTML or Markdown report. The user can leave feedback directly on the Artifact—similar to commenting on a collaborative document—and the agent incorporates the input without breaking its execution flow.⁸

The Persistent Knowledge Base and Configuration Standards

To prevent the AI from losing context over long-horizon projects, Antigravity introduces a "Persistent Knowledge Base".⁸ This utilizes a localized directory structure to store long-term memory and cross-tool configurations, ensuring the agents remain anchored to the project's specific financial methodologies.

The orchestration behavior is governed by a strict hierarchy of configuration files:

Configuration File	Scope and Priority	Primary Function in Financial Architecture
GEMINI.md	Highest user priority; Antigravity-specific.	Defines core orchestration loops, strictly enforces the generation of financial Artifacts, and configures the IDE's specific automated behaviors.

AGENTS.md	Cross-tool standard; lower priority foundation.	Defines the universal technical stack (e.g., Python 3.12, Pandas), testing requirements (mandating TDD for math functions), and global safety guardrails prohibiting credential hardcoding.
.agent/rules/	Workspace supplements; highly specific.	Modular files (e.g., dcf-rules.md, ibkr-parsing.md) that separate concerns, allowing the agent to dynamically load specific instructions only when engaging in relevant tasks.

This file structure ensures that the agents adhere to a unified set of principles, preventing hallucinatory deviations from the established financial objective.¹⁶ By explicitly defining constraints within AGENTS.md, the architect ensures that any agent spawned within the project inherently understands the formatting requirements for the final Friday email and the strict prohibition against autonomous trade execution.¹⁶

Multi-Agent Topology and Orchestration Best Practices

The ambition of identifying undervalued market opportunities and monitoring a dynamic portfolio necessitates a modular, decentralized intelligence architecture. Relying on a singular language model to parse Securities and Exchange Commission (SEC) filings, compute complex cash flow models, and analyze macroeconomic commodity cycles inevitably leads to context saturation and logical collapse.⁵ A highly specialized ensemble approach is required.

Progressive Disclosure and Agent Skills

To solve the issue of context rot, the architecture must implement Progressive Disclosure via Agent Skills. A Skill packages raw Python functions, system prompts, and domain context into a single, modular deployable unit.⁵ Rather than forcing the model to memorize the entirety of the financial codebase at the start of a session, the model is exposed only to a lightweight menu of metadata.⁵

The directory structure of an Antigravity Skill is strictly standardized. It revolves around a mandatory SKILL.md file, accompanied by optional scripts/ or references/ directories.⁵ The SKILL.md file utilizes YAML frontmatter to define the skill's metadata and a precise description.¹⁹ The agent initially loads only these metadata descriptions (Level 1 loading). When the user's intent matches the description—for example, invoking the ibkr-portfolio-ingestion skill—the agent dynamically retrieves the heavy procedural instructions and Python execution pathways (Level 2 loading) solely for the duration of that task.⁵ This structural design guarantees that an agent engaged in evaluating the marginal cost of lithium production is not burdened with the contextual overhead of parsing an XML API response from a brokerage account.

The Orchestrator-Worker Topology and The Critic Agent

The optimal design for the financial engine is an Orchestrator-Worker pattern, where a central supervisor decomposes the high-level goal, assigns subtasks to specialized worker agents, and consolidates their results.² This topology minimizes the action space for any individual agent, drastically reducing the probability of catastrophic compounding errors during autonomous execution.

To mathematically counteract the aforementioned optimistic bias inherent in LLMs, the architecture must implement a Generator-Critic loop.²⁰ In this paradigm, when the Analyst Agent (the generator) extracts a bullish five-year revenue growth projection from an earnings transcript, the data is not immediately piped into the valuation algorithm. Instead, the Critic Agent is triggered. The Critic's system prompt is explicitly adversarial; it is mandated to cross-reference the Analyst's optimistic parameters against broader industry headwinds, geopolitical supply chain risks, and historical base rates.

Empirical research on multi-agent systems demonstrates that implementing a self-refine Critic pattern improves complex mathematical reasoning solve rates from 22.1% to 59.0%.²¹ By enforcing this adversarial consensus mechanism, the system computationally demands a wider margin of safety before finalizing the parameter estimates, directly offsetting the native positive skew of the foundation models.³ If the Critic detects logical fallacies or unsupported optimism, it returns the analysis to the generator with specific feedback, demanding recalculation prior to approval.²¹

Mitigating Thesis Drift via Persistent Memory

A common behavioral pitfall in active satellite investing is "thesis drift"—the psychological rationalization of holding a depreciating asset by subconsciously altering the original rationale for the investment.²³ An autonomous system must rigorously enforce discipline by computationally detecting and flagging this cognitive drift.

The architecture utilizes the persistent memory capabilities of the IDE to maintain a local, hidden directory containing strictly managed files that act as the long-term memory of the system.²⁵ When a new equity is added to the 5% satellite portfolio, the orchestrator generates a

thesis_log.md entry detailing the precise factor anomalies, estimated real value, and qualitative catalysts that justified the purchase.²⁵ During the weekly monitoring cycle, the Critic Agent is instructed to cross-reference the latest fundamental data against this persistent historical record. If a company originally purchased for high-growth capital expenditure shifts its strategy to defensive dividend yields, the Critic Agent computationally identifies the deviation and alerts the user to the thesis drift in the weekly report, recommending an objective re-evaluation of the position.²⁵ To prevent the memory files from growing indefinitely and consuming excess tokens, the agents are instructed to continuously compress and synthesize older data, maintaining a strict line limit on the strategic memory files.²⁵

Advanced Financial Routing and Valuation Methodologies

The determination of an asset's "estimated real value" cannot be achieved through a monolithic algorithm. Different asset classes are governed by entirely disparate economic physics. A Discounted Cash Flow (DCF) model is fundamentally incompatible with non-yielding physical commodities like gold or lithium, and it is computationally exhausting to apply individually to the thousands of underlying holdings in a broad market ETF.²⁹ The multi-agent system must employ an intelligent Router Pattern that analyzes the asset type and dynamically delegates parameter estimation to the appropriate deterministic mathematical model.⁵

Equity Valuation: Parameterized DCF and Monte Carlo Ensembles

For mature, cash-generating equities, the system relies on the Discounted Cash Flow framework. However, the fundamental vulnerability of algorithmic DCF modeling is extreme sensitivity; a fractional modification to the terminal growth rate or the Weighted Average Cost of Capital (WACC) can drastically alter the output. Therefore, the LLM is strictly prohibited from executing the calculation itself. Instead, it serves solely as an unstructured data processor, extracting Base, Bull, and Bear percentage estimations for revenue growth, operating margins, and capital expenditures from qualitative texts like 10-K filings.

These estimations are piped into a deterministic Python function that executes the core DCF formula:

$$V_0 = \sum_{t=1}^n \frac{FCF_t}{(1+WACC)^t} + \frac{TV}{(1+WACC)^n}$$

To further insulate the portfolio against mathematical fragility, this DCF model is encapsulated within a Monte Carlo simulation. Rather than calculating a single intrinsic value, the Python engine utilizes the agent's Base, Bull, and Bear parameters to define a probability distribution:

$$FCF_t \sim \mathcal{N}(\mu_{FCF}, \sigma_{FCF}^2)$$

The Critic Agent dynamically adjusts the standard deviation (σ) based on macroeconomic

uncertainty extracted from recent news sentiment. The simulation iterates thousands of times, producing a probabilistic bell curve of fair value. This enables the weekly report to present the statistical probability of an asset trading at a 20% discount to its intrinsic value, rather than relying on a singular, brittle price target.

Physical Market Modeling: Industrial and Precious Metals

Non-yielding commodities necessitate specialized macrofinancial models based on physical market balances.²⁹

For industrial metals like lithium, the pricing mechanism is fundamentally anchored to the structural deficit and the marginal cost of production curve. By 2026, demand modeling must account not only for electric vehicle penetration but also for the proliferation of grid-forming storage required to stabilize electrical grids against the massive energy load of AI data centers.³¹ The Analyst Agent is routed to ingest dense mineral intelligence reports, extracting these specific demand vectors. The deterministic Python algorithm calculates the structural deficit: $D_t = (EV_t \times C_{EV}) + (ESS_t \times C_{ESS}) - S_t$. If the projected deficit (D_t) is sustained, the system cross-references the current spot price against the marginal cost required to incentivize new extraction capacity, generating objective metrics regarding price sustainability.³¹

Conversely, the valuation of precious metals like gold relies on an aggregate macro-regime model. The agent analyzes unstructured data regarding structural central bank accumulation, fiat debasement risk, and geopolitical hedging.²⁹ By quantifying these qualitative sentiment indicators and mapping them against historical correlation matrices involving real interest rates, the Python backend generates a probability-weighted target price band.

Broad ETFs: Macro-Regime Aggregation

For massive aggregates like Vanguard Total World Stock (VT) or specific factor ETFs, the system assesses the "Macro Regime" to determine if the current economic climate favors the specific exposure. This relies on trend analysis tracking aggregate S&P 500 earnings revisions, broad inflation stickiness, and labor market slack. The agent acts as a macro-strategist, classifying the current environment into distinct regimes (Expansion, Slowdown, Recession, Recovery). If the system identifies a shift into a stagflationary environment, it automatically alerts the user regarding the potential vulnerability of their broad equity exposure, analyzing the correlation between the passive core and the active satellite bets.

Governed Data Ingestion Pipelines

The efficacy of the analytical models is entirely dependent on the integrity and timeliness of the underlying data. The system requires governed, read-only data bridges to extract both personal portfolio states and broader market fundamentals securely.

The Interactive Brokers Integration

To safely monitor the user's active holdings without introducing the catastrophic tail risk of an autonomous execution bot erroneously liquidating a portfolio, the system utilizes the Interactive Brokers Flex Web Service.³³ While high-frequency trading mandates complex socket connections via the IB Gateway, a weekly reporting pipeline is optimally served by automated, end-of-day static reports, eliminating the need for constant server maintenance and daily authentication restarts.³⁶

As of the recent architectural updates, Interactive Brokers consolidated its endpoints under a unified Web API umbrella, utilizing OAuth 2.0 for secure authentication.³³ The architecture requires the user to configure a "Flex Query" within the IBKR Client Portal, explicitly defining the required output columns (Ticker, Position, Cost Basis, Currency, and Unrealized P&L).³⁴ A long-lived Flex Web Service Token is generated. The Python ingestion module utilizes standard HTTP libraries to poll the ndcdyn endpoints, transmitting the Token and Query ID to retrieve the dataset.³⁵ As previously established, this ingestion layer must employ strict CSV parsing protocols to handle comma-separator anomalies, converting the raw export into a validated internal format before exposing it to the analytical agents.¹

Financial Modeling Prep API Integration

For historical financial statements, real-time equity quotes, and cross-asset normalization, the system integrates the Financial Modeling Prep (FMP) API.³⁸ Evaluated as a premier developer-friendly integration layer in 2026, FMP provides institutional-grade multi-asset infrastructure with highly consistent JSON schemas.³⁹ This schema consistency drastically reduces the translation friction required when the orchestrator agent switches contexts between pulling a 10-year cash flow statement for a technology stock and querying real-time forex rates.³⁹ The free tier permits 250 API requests daily, providing sufficient bandwidth for a strictly capped 5% satellite portfolio running batch processes on a weekly schedule.⁴⁰

Regulatory Compliance and Security Frameworks (2026)

The deployment of autonomous logic in financial environments introduces complex legal, ethical, and cybersecurity challenges that must be preemptively mitigated. In 2026, the regulatory landscape demands rigorous compliance architectures for AI systems, heavily governed by the enforcement phase of the European Union's Artificial Intelligence Act (August 2026) and stringent interpretations of the General Data Protection Regulation (GDPR).⁴² While a personal portfolio monitor may operate below the threshold of enterprise liability, adhering to these institutional standards is a vital best practice for operational security.

The AEPD Guidance on Agentic AI and Memory Management

In February 2026, the Spanish Data Protection Authority (AEPD) released comprehensive guidance specifically addressing the deployment of "Agentic AI".⁴⁵ The AEPD explicitly

highlights the vulnerabilities introduced by an agent's memory architecture, distinguishing between working memory (utilized for immediate task execution) and management memory (utilized for long-term learning and context retention).⁴⁸ If an agent indefinitely stores historical portfolio balances, behavioral traits, and specific financial decisions in its persistent knowledge base without strict boundaries, it violates the core GDPR principles of purpose limitation and data minimization.⁴⁵

To achieve compliance, the system must implement automated memory lifecycle management.⁴⁹ Long-term memory components must be computationally restricted to store only aggregated factor methodologies and anonymized thesis rationale. The system must systematically erase specific transactional data and account balances from the agent's operational memory immediately after the weekly Friday report is generated, ensuring that no sensitive personal data persists within the AI's contextual vectors.⁴⁵

The "Rule of 2" and Human-in-the-Loop Oversight

A cornerstone of the AEPD's 2026 guidance is the application of the "Rule of 2" for risk management. The rule stipulates that an autonomous system must never simultaneously combine three specific risk factors: (i) processing uncontrolled input, (ii) accessing sensitive information, and (iii) performing autonomous actions.⁴⁸

The proposed multi-agent system inherently processes uncontrolled external inputs (macroeconomic news feeds, unstructured earnings transcripts) and analyzes highly sensitive internal information (the user's private Interactive Brokers portfolio data and total net worth calculations). Therefore, to strictly adhere to the Rule of 2, the system must be permanently stripped of the third factor: it is completely prohibited from executing autonomous trading actions.⁴⁸

The ultimate fail-safe is the structural enforcement of Human-in-the-Loop oversight.⁴⁷ The objective of the multi-agent system is exclusively analytical synthesis. By compiling the valuations, the margin of safety statistical distributions, and the thesis drift warnings into a static HTML email report, the architecture guarantees that human agency remains the sole arbiter of capital deployment. This paradigm entirely neutralizes the threat of catastrophic autonomous execution while retaining the massive analytical leverage provided by the artificial intelligence engine.⁵⁰

Phased Implementation Blueprint

Transitioning this theoretical architecture into a production-ready application requires a methodical, step-by-step engineering roadmap. A fundamental best practice in deploying multi-agent systems is to explicitly avoid leading with a multi-agent swarm; coordination overhead often dominates the quality gain if a single-agent system has not yet hit its performance ceiling.⁵³ Therefore, the implementation must progress from simple verification to complex orchestration.

Phase 1: Zero-Code Base and Governable Bridges

The foundation begins with establishing secure data ingestion pipelines without introducing any AI logic.

- **Brokerage Handshake:** Configure the Interactive Brokers Client Portal to generate a Flex Web Service Token. Define a specific Query ID containing the necessary portfolio state vectors (Ticker, Position, Cost Basis).³⁴
- **Market Data Connector:** Obtain API credentials for Financial Modeling Prep to enable historical fundamental ingestion.³⁸ Use simple tools like cURL or Postman to verify endpoint connectivity.⁴⁰
- **Secret Management:** Establish a local credential vault, utilizing environment files (.env) strictly excluded from version control, ensuring no API keys are exposed to the AI agents.⁵⁴
- **Deterministic Validation:** Write a standalone Python script that fetches the IBKR data, parses it securely using explicit comma-separator logic, and prints a simple text summary to the console.

Phase 2: Single-Agent Portfolio Summarizer

Introduce the AI strictly for summarization tasks to establish the reporting baseline.

- **Environment Configuration:** Install the Google Antigravity IDE. Define the fundamental rules within the AGENTS.md and GEMINI.md files, explicitly barring the system from writing trading scripts.¹⁶
- **The Controller Agent:** Construct a single Python script that passes the validated portfolio data array from Phase 1 to a model via the Gemini API.
- **Prompt Engineering:** Instruct the agent to analyze the array and write a two-sentence objective summary for each asset based on recent news. Output the result to an HTML file.

Phase 3: Antigravity Skills Integration and Algorithmic Core

Develop the deterministic mathematical engine and package it for agent discovery.

- **Valuation Logic:** Develop pure Python functions for the Discounted Cash Flow model, the Monte Carlo simulation wrapper, and the Industrial Deficit capacity calculation. Implement Test-Driven Development (TDD) to verify that the modules correctly handle edge cases.
- **Skill Creation:** Package these Python algorithms into Antigravity Skills. Construct the specific <skill-name>/ directories and write the SKILL.md YAML frontmatter, allowing the agents to dynamically load the fundamental analysis tools via Progressive Disclosure.¹⁹

Phase 4: Multi-Agent Swarm and The Critic

Transition from a linear script to an Orchestrator-Worker multi-agent topology.

- **Agent Personas:** Define the specific instructions for the Scout (discovery), the Analyst (parameter extraction), and the Critic (bias mitigation and verification).²¹
- **The Generator-Critic Loop:** Implement the adversarial consensus mechanism. Ensure the Critic Agent forces recalculations if the Analyst's parameter estimates exhibit optimistic bias or deviate from historical base rates.²¹
- **Thesis Tracking:** Initialize the hidden .antigravity/ directory. Instruct the Critic Agent to manage the thesis_log.md file, computationally enforcing line limits by synthesizing historical rationale to track thesis drift without violating data minimization principles.²⁵

Phase 5: Full Automation and Synthesis

The final phase focuses on scheduling and the human-computer interface.

- **Template Engine:** Utilize a library such as Jinja2 to parse the structured JSON outputs from the multi-agent swarm into a responsive, visually clear HTML email template. Include tabular comparisons and statistical probability distributions for margin of safety metrics.
- **Scheduling:** Deploy a Unix Cron job or a lightweight serverless function to autonomously trigger the Antigravity workflow orchestration script every Friday afternoon, culminating in the secure SMTP transmission of the weekly portfolio briefing.

Conclusion

Constructing a proprietary, automated investment engine to manage a high-conviction satellite portfolio is a complex engineering endeavor that demands rigorous adherence to software architecture principles, financial theory, and regulatory compliance. The multi-agent architecture delineated in this report solves for the critical failure modes of both retail investing and early-generation AI deployment.

By leveraging Google Antigravity as an orchestration platform, the system successfully abstracts the overwhelming volume of market data into clear, actionable intelligence using verifiable Artifacts. It structurally enforces investment discipline by utilizing the Generator-Critic pattern, neutralizing the inherent optimistic bias of generative models by demanding adversarial consensus. Furthermore, by strictly adhering to a read-only data bridge with Interactive Brokers, implementing automated memory lifecycle management to prevent excessive data retention, and restricting output to a comprehensive weekly email report, the architecture fully satisfies the rigorous Human-in-the-Loop and "Rule of 2" compliance frameworks outlined by European data protection authorities in 2026. This blueprint provides the foundation for an analytical engine capable of operating autonomously in the background, continuously executing objective fundamental analysis to identify the margin of safety required for sustainable market outperformance.

Obras citadas

1. IBKR error [1001]: Statement could not be generated at this time. :
r/interactivebrokers, fecha de acceso: mayo 14, 2026,
https://www.reddit.com/r/interactivebrokers/comments/1t01qem/ibkr_error_1001_statement_could_not_be_generated/
2. Why Multi-Agent Systems Need Real-Time Context in 2026 - Solace, fecha de acceso: mayo 14, 2026,
<https://solace.com/blog/analysts-say-mas-needs-real-time-context-eda/>
3. How Effectively Can Current LLMs Analyze Macrofinancial Issues ..., fecha de acceso: mayo 14, 2026,
<https://www.imf.org/-/media/files/publications/wp/2026/english/wpiea2026035-source-pdf.pdf>
4. Predicting Biased Human Decision-Making with Large Language Models in Conversational Settings - arXiv, fecha de acceso: mayo 14, 2026,
<https://arxiv.org/html/2601.11049v1>
5. Authoring Google Antigravity Skills, fecha de acceso: mayo 14, 2026,
<https://codelabs.developers.google.com/getting-started-with-antigravity-skills>
6. How to Build Multi-Agent Systems: Complete 2026 Guide - DEV Community, fecha de acceso: mayo 14, 2026,
<https://dev.to/eira-wexford/how-to-build-multi-agent-systems-complete-2026-guide-1io6>
7. Building multi-agent systems will be a must-have PM skill in 2026. - Aakash Gupta, fecha de acceso: mayo 14, 2026,
<https://aakashgupta.medium.com/building-multi-agent-systems-will-be-a-must-have-pm-skill-in-2026-30e52fd732cb>
8. Build with Google Antigravity, our new agentic development platform ..., fecha de acceso: mayo 14, 2026,
<https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/>
9. Complete Guide to Google Antigravity (2026) | Tutorial & Documentation, fecha de acceso: mayo 14, 2026, <https://antigravity.codes/tutorial>
10. Google Antigravity: The Agentic OS That Replaces the Code Editor | JavaScript in Plain English, fecha de acceso: mayo 14, 2026,
<https://javascript.plainenglish.io/liftoff-why-google-antigravity-isnt-just-an-editor-it-s-an-agentic-operating-system-for-code-84f35cee5534>
11. Tutorial : Getting Started with Google Antigravity | by Romin Irani - Medium, fecha de acceso: mayo 14, 2026,
<https://medium.com/google-cloud/tutorial-getting-started-with-google-antigravity-b5cc74c103c2>
12. Antigravity Codes | 1,500+ MCP Servers, AI Rules & Workflows for Antigravity, Cursor & Windsurf, fecha de acceso: mayo 14, 2026, <https://antigravity.codes/>
13. I tried Google's new Antigravity IDE so you don't have to (vs Cursor/Windsurf) - Reddit, fecha de acceso: mayo 14, 2026,
https://www.reddit.com/r/ChatGPTCoding/comments/1p35bdl/i_tried_googles_new_antigravity_ide_so_you_dont/
14. Google Antigravity Tutorial: Build a Finance Risk Dashboard - DataCamp, fecha de

- acceso: mayo 14, 2026,
<https://www.datacamp.com/tutorial/google-antigravity-tutorial>
15. How to Use Google Antigravity to Build Software with AI Agents - Stork.AI, fecha de acceso: mayo 14, 2026,
<https://www.stork.ai/blog/googles-ai-coder-is-here-its-scary-good>
 16. Guide with AGENTS.md & Examples (2026) - Antigravity Rules, fecha de acceso: mayo 14, 2026, <https://antigravity.codes/blog/user-rules>
 17. AGENTS.md Guide: Cross-Tool Rules for Antigravity (2026), fecha de acceso: mayo 14, 2026, <https://antigravity.codes/blog/antigravity-agents-md-guide>
 18. Tutorial : Getting Started with Google Antigravity Skills, fecha de acceso: mayo 14, 2026,
<https://medium.com/google-cloud/tutorial-getting-started-with-antigravity-skills-864041811e0d>
 19. Antigravity Skills Setup Guide: Install & Configure (2026), fecha de acceso: mayo 14, 2026, <https://antigravity.codes/blog/antigravity-skills-setup-guide>
 20. AI Agent Swarms: When Multi-Agent Systems Are Worth It - Codebridge, fecha de acceso: mayo 14, 2026,
<https://www.codebridge.tech/articles/ai-agent-swarms-when-multi-agent-systems-create-value-and-when-they-just-add-complexity>
 21. AI Agent Architecture: Design Patterns for Scalable Agent Systems - Codewave, fecha de acceso: mayo 14, 2026,
<https://codewave.com/feeds/blog/ai-agent-architecture-patterns>
 22. AI-Supervisor: Autonomous AI Research Supervision via a Persistent Research World Model - arXiv, fecha de acceso: mayo 14, 2026,
<https://arxiv.org/html/2603.24402v1>
 23. Modeling Driver Distraction - mediaTUM, fecha de acceso: mayo 14, 2026,
<https://mediatum.ub.tum.de/doc/1341249/1341249.pdf>
 24. Organising for power change - Transformative business models for the energy transition - DRIFT, fecha de acceso: mayo 14, 2026,
https://drift.eur.nl/app/uploads/2021/03/Proka-A_Organising-for-power-change_FINAL.pdf
 25. How I gave Google Antigravity a real long-term memory : r/google_antigravity - Reddit, fecha de acceso: mayo 14, 2026,
https://www.reddit.com/r/google_antigravity/comments/1tbvmnb/how_i_gave_google_antigravity_a_real_longterm/
 26. How to Set Up and Use Google Antigravity - Codecademy, fecha de acceso: mayo 14, 2026,
<https://www.codecademy.com/article/how-to-set-up-and-use-google-antigravity>
 27. Auto-claude-code-research-in-sleep/README.md at main - GitHub, fecha de acceso: mayo 14, 2026,
<https://github.com/wanshuiyin/Auto-claude-code-research-in-sleep/blob/main/README.md>
 28. Continuous Machine Learning for Cooperative ... - DSpace@NTUA, fecha de acceso: mayo 14, 2026,

- <https://dspace.lib.ntua.gr/xmlui/bitstream/handle/123456789/64319/main.pdf?sequence=1&isAllowed=y>
29. Commodity Markets Outlook -- April 2026, fecha de acceso: mayo 14, 2026, <https://openknowledge.worldbank.org/server/api/core/bitstreams/a384bcad-abc5-4b9b-9c1d-35f374581024/content>
 30. 2026 Long-Term Capital Market Assumptions - J.P. Morgan Asset Management, fecha de acceso: mayo 14, 2026, <https://am.jpmorgan.com/content/dam/jpm-am-aem/americas/us/en/institutional/insights/portfolio-insights/lcma-full-report.pdf>
 31. ICIS Supply and Demand Database, fecha de acceso: mayo 14, 2026, <https://www.icis.com/explore/services/supply-and-demand-database/>
 32. Lithium Price Outlook: Trend Insights & Forecast - Farmonaut, fecha de acceso: mayo 14, 2026, <https://farmonaut.com/mining/lithium-price-outlook-2026-trend-insights-forecast>
 33. Introduction | IBKR API | IBKR Campus, fecha de acceso: mayo 14, 2026, <https://www.interactivebrokers.com/campus/ibkr-api-page/getting-started/>
 34. Flex Web Service - IBKR Campus, fecha de acceso: mayo 14, 2026, <https://www.interactivebrokers.com/campus/ibkr-api-page/flex-web-service/>
 35. Flex Web Service - IBKR Campus, fecha de acceso: mayo 14, 2026, <https://www.interactivebrokers.com/campus/glossary-terms/flex-web-service/>
 36. Advantages of Learning Interactive Brokers Python API, fecha de acceso: mayo 14, 2026, <https://www.interactivebrokers.com/campus/ibkr-quant-news/advantages-of-learning-interactive-brokers-python-api/>
 37. Web API Documentation - IBKR Campus, fecha de acceso: mayo 14, 2026, <https://www.interactivebrokers.com/campus/ibkr-api-page/webapi-doc/>
 38. Pricing | Financial Modeling Prep | FMP, fecha de acceso: mayo 14, 2026, <https://site.financialmodelingprep.com/developer/docs/pricing>
 39. Best Developer-Friendly Financial Data APIs for Bu... | FMP, fecha de acceso: mayo 14, 2026, <https://site.financialmodelingprep.com/how-to/how-to-evaluate-developerfriendly-financial-data-apis-and-crossasset-integration-strategies>
 40. Five Key Steps to Integrate FMP APIs Without Writing a Single Line of Code, fecha de acceso: mayo 14, 2026, <https://site.financialmodelingprep.com/how-to/five-key-steps-to-integrate-fmp-apis-without-writing-a-single-line-of-code>
 41. FAQs - Financial Modeling Prep API | FMP, fecha de acceso: mayo 14, 2026, <https://site.financialmodelingprep.com/faqs>
 42. What to expect in 2026 for GDPR and data protection - VinciWorks, fecha de acceso: mayo 14, 2026, <https://vinciworks.com/blog/what-to-expect-in-2026-for-gdpr-and-data-protection/>
 43. AI Agent Compliance for Financial Services (2026) - Fin AI, fecha de acceso: mayo 14, 2026, <https://fin.ai/learn/evaluate-ai-agent-compliance-financial-services>

44. EU AI Act Compliance: How to Prepare for 2026, fecha de acceso: mayo 14, 2026, <https://securityboulevard.com/2026/05/eu-ai-act-compliance-how-to-prepare-for-2026/>
45. Spanish Supervisory Authority Issues Detailed Guidance on Agentic AI and GDPR Compliance | Inside Privacy, fecha de acceso: mayo 14, 2026, <https://www.insideprivacy.com/artificial-intelligence/spanish-supervisory-authority-issues-detailed-guidance-on-agentic-ai-and-gdpr-compliance/>
46. AEPD publishes guidance on Agentic Artificial Intelligence from a data protection perspective - A&O Shearman, fecha de acceso: mayo 14, 2026, <https://www.aoshearman.com/en/insights/ao-shearman-on-data/aepd-publishes-guidance-on-agentic-artificial-intelligence-from-a-data-protection-perspective>
47. AGENTIC ARTIFICIAL INTELLIGENCE DATA PROTECTION, fecha de acceso: mayo 14, 2026, <https://www.aepd.es/en/guides/agentic-artificial-intelligence.pdf>
48. AEPD Publishes Guidance On Agentic Artificial Intelligence From A Data Protection Perspective | A&O Shearman - JD Supra, fecha de acceso: mayo 14, 2026, <https://www.jdsupra.com/legalnews/aepd-publishes-guidance-on-agentic-6266860/>
49. Agentic AI and data protection: guidance from the Spanish AEPD - Linklaters - Tech Insights, fecha de acceso: mayo 14, 2026, <https://techinsights.linklaters.com/post/102mk6z/agentic-ai-and-data-protection-guidance-from-the-spanish-aepd>
50. TTLF Working Papers - Stanford Law School, fecha de acceso: mayo 14, 2026, <https://law.stanford.edu/wp-content/uploads/2026/04/TTLF-WP-150-Yazgan.pdf>
51. A Closer Look at Spain's Agentic AI and Data Protection Guidance | NAVEX - JDSupra, fecha de acceso: mayo 14, 2026, <https://www.jdsupra.com/legalnews/a-closer-look-at-spain-s-agentic-ai-and-5195343/>
52. What is Agentic AI? A Primer for Legal and Privacy Teams, fecha de acceso: mayo 14, 2026, <https://www.privacyworld.blog/2025/06/what-is-agentic-ai-a-primer-for-legal-and-privacy-teams/>
53. Agent Architecture Patterns: 2026 Taxonomy Guide - Digital Applied, fecha de acceso: mayo 14, 2026, <https://www.digitalapplied.com/blog/agent-architecture-patterns-taxonomy-2026>
54. runlayer - PyPI, fecha de acceso: mayo 14, 2026, <https://pypi.org/project/runlayer/>
55. Antigravity + Claude Opus 4.6 = Incredible : r/google_antigravity - Reddit, fecha de acceso: mayo 14, 2026, https://www.reddit.com/r/google_antigravity/comments/1r74ikf/antigravity_claude_opus_46_incredible/